PRODUCT REQUIREMENTS DOCUMENT

Plataforma de generación y publicación de Voicebots (voz + chat) con RAG

0) Resumen ejecutivo

Se construirá una plataforma que permita a usuarios internos crear y publicar **voicebots** configurables (nombre, idioma, voz, master prompt y knowledge base). Cada bot publicado genera un **link público** con una interfaz estilo *taina.gob.do* (voz-first) e incluye además **interfaz de chat**.

Los usuarios finales acceden a la conversación **mediante login obligatorio OAuth** (Google/ Microsoft). Los admins usan **GOB-ID**. El bot responde únicamente con conocimiento obtenido del **knowledge base** (RAG *grounded-only*). Si no hay evidencia, responde “no sé”.

Se desea que en las configuraciones los voicebots puedan ser configurables para integraciones futuras vía function calls / MCP.

1) Objetivos

- Permitir crear y operar voicebots sin desarrollo a medida.
- Entregar una experiencia de voz de baja latencia con streaming y controles tipo call.
- Habilitar RAG con knowledge base editable (solo .txt) y política **top-k**.
- Proveer observabilidad por eventos y logs para dashboards futuros.

2) Alcance

Incluido (In-scope)

- Portal admin para crear/editar/publicar bots.
- Interfaz pública de conversación: voz + chat.
- OAuth requerido para conversar (Google o Microsoft).
- Pipeline: STT → embeddings → búsqueda vectorial → LLM → TTS, com streaming.
- Knowledge base editable **solo .txt**.
- Logs de conversación (texto) y eventos almacenados en BD relacional.
- Configuración de proveedores por API Keys (App Settings).
- Despliegue **SaaS o Standalone** (containers en infraestructura de terceros).
- Preparación para integraciones futuras vía function calls / MCP.

Excluido (Out-of-scope)

- Pagos in-app (gestión externa).
- Soporte/mesa de ayuda dentro del producto (se maneja aparte).
- Gestión y redacción de PII, “right to be forgotten”.
- Versionado de master prompt, variables de prompt.
- Retries automáticos hacia proveedores (sin reintentos).

3) Personas, roles y autenticación

3.1 Identidad y autenticación

- **Admins (Portal de Gestión):** GOB-ID (JWT en localStorage).

- **Usuarios finales (Interfaz pública):** OAuth obligatorio con Google o Microsoft para enviar mensajes (voz o chat). Sin login solo pueden ver la pantalla inicial.

3.2 Roles (RBAC) — Portal (GOB-ID)

Acciones se asignan a nivel de Tenant/Organización y/o Bot.

R1 — Platform Super Admin (opcional)

- Admin global (tenants, settings globales, auditoría completa).

R2 — Tenant Admin

- CRUD de bots del tenant.
- Gestión de usuarios internos y roles.
- App Settings (API Keys, selección de proveedores).
- Acceso completo a logs y eventos del tenant.

R3 — Bot Manager

- Configura bot: nombre, idioma, voz, master prompt.
- Ejecuta modo “Probar”.
- Ve logs/eventos del bot.
- Publicar si tiene permiso explícito bot:publish.

R4 — Knowledge Base Editor

- Gestiona KB: subir/editar/borrar .txt, reindexar.
- No puede editar master prompt, proveedores ni publicar.

R5 — Analyst (Read-only analytics)

- Solo lectura de eventos, métricas y reportes.

R6 — Auditor (Read-only config + conversaciones)

- Lectura de configuración + transcripciones + eventos.

3.3 Roles — Interfaz pública (OAuth)

P1 — Authenticated Public User

- Puede iniciar conversación por voz y/o chat.
- Sus interacciones se registran (texto + eventos).

P0 — Visitor

- Puede ver landing y CTA de login.
- No puede enviar mensajes.

4) User journeys principales

4.1 Admin crea y publica un bot

1. Login GOB-ID
2. Crear bot → definir nombre + idioma + master prompt
3. Configurar voz (catálogo/ID) y proveedor (por API Keys en App Settings)
4. Subir knowledge base (.txt)
5. Ejecutar “Probar” (pre-publicación)
6. Publicar → se genera link público y UI base

4.2 Usuario final conversa (voz + chat)

1. Abre link público
2. Se muestra UI estilo Taina + CTA “Inicia sesión”
3. OAuth (Google/Microsoft)
4. Conversa por voz o chat
5. Respuesta en streaming (texto) + audio (TTS) con estados y feedback

5) Requisitos funcionales

5.1 Gestión de bots (Portal)

RF-1 Crear/editar/eliminar bot.
RF-2 Configurar:

- Nombre del bot (display en UI).
- Idioma del bot.
- Master prompt (se sobreescribe; sin versionado; sin variables).
- Voz(es) disponibles y voz activa (por proveedor).
- Parámetros de retrieval: top_k (configurable por bot con default global).
  RF-3 Publicar/despublicar bot (genera/invalida link).
  RF-4 Permisos por RBAC para cada acción.

5.2 Knowledge base (solo .txt)

RF-5 CRUD de documentos .txt por bot:

- Subir (upload) .txt
- Editar (reemplazar contenido)

- Borrar
- Reindexar manual
  RF-6 Ingesta:
- Chunking del texto y generación de embeddings.
- Persistencia en vector DB.
  RF-7 Retrieval:
- Política **top-k**.
- retrieval_hit si hay resultados en top-k (y opcionalmente si superan umbral configurable).
  RF-8 Grounded-only:
- El LLM **solo** responde con evidencia de los documentos recuperados.
- Si retrieval_miss ⇒ respuesta “No tengo información en mi base de conocimiento para responder eso.”

5.3 Interfaz pública (voz-first + chat)

Basada en el layout observado (dark full-screen, logo arriba izquierda, indicador central, barra inferior con mic y botón cerrar).

RF-9 UI estilo “call”:

- Pantalla full-screen oscura.
- Logo/nombre del bot arriba izquierda.
- Indicador central (p.ej. puntos) para “listening/thinking”.
- Texto de estado (ej. “X te está escuchando…”).
- Barra inferior con:
  - Mic (start/stop).
  - Selector (dropdown) para opciones rápidas (p.ej. idioma/voz, si se permite al usuario final).
  - Botón de chat (abre panel).
  - Botón “Cerrar” (termina sesión).

RF-10 Chat integrado:

- Panel lateral o modal que muestra la conversación textual.
- Lo hablado se transcribe y aparece como mensaje del usuario.
- La respuesta aparece en streaming como texto y (si aplica) como audio.

RF-11 Login obligatorio para conversar:

- Sin OAuth: UI visible pero no permite enviar.
- Con OAuth: habilita input de chat y mic.

RF-12 Feedback mientras carga:

- Indicador visual + mensaje.
- Opcional: sonido “typing/teclado” durante thinking.

5.4 Experiencia de voz: turn-taking, streaming, formatos y red

RF-13 Turn-taking:

- VAD (detección de voz) para inicio/fin de turno.
- Silencios y timeouts configurables:
  - silence_end_ms, max_utterance_ms, no_speech_timeout_ms.
- Barge-in:
  - Si el usuario habla durante TTS, se corta el audio y se toma el turno del usuario.
  - Emitir evento barge_in.

RF-14 Streaming end-to-end:

- STT parcial → UI/Chat muestra transcripción parcial.
- LLM streaming → UI/Chat muestra respuesta parcial.
- TTS streaming → iniciar reproducción sin esperar respuesta completa.
- Evento tts_first_audio cuando se reproduce el primer audio.

RF-15 Formatos y compatibilidad:

- Considerar **WebRTC** para baja latencia.
- Codec: Opus (vía WebRTC) o equivalente.
- Resample si STT requiere formato específico.
- Compatibilidad mobile: permisos, fallback si no hay soporte/permiso.

RF-16 Manejo de red:

- Estados reconnecting y recuperación del canal.
- Manejo de jitter.
- Fallback a modo text_only_fallback cuando audio no sea viable.

5.5 Pipeline conversacional (RAG)

RF-17 Flujo por turno (voz):

1. Captura audio (WebRTC/WebAudio)
2. STT (ej. AssemblyAI u otro)
3. Transcripción final
4. Embedding de la transcripción
5. Búsqueda vectorial top-k en KB
6. Construcción de prompt (master prompt + contexto top-k + pregunta)
7. LLM responde (streaming)
8. TTS (ej. ElevenLabs u otro) con streaming
9. Persistencia: mensajes + eventos

RF-18 Flujo por turno (chat):

- Igual que voz, sustituyendo STT por texto directo del usuario.

RF-19 Sin reintentos a proveedores:

- Si falla STT/LLM/TTS ⇒ error user-facing + evento de falla.
- No se ejecutan retries automáticos.

5.6 Master prompt y pruebas

RF-20 Master prompt:

- Se edita y se **sobreescribe**.
- No versionado.
- No variables.
- Guardrails/políticas deben vivir en el prompt.

RF-21 Modo “Probar” antes de publicar (obligatorio):

- Conversación de prueba en portal, sin exponer públicamente.
- Debe mostrar (interno):
    - Transcripción
    - top-k recuperado (IDs/títulos de docs o fragmentos)
    - respuesta LLM
    - latencias por etapa
- Publicar solo tras prueba manual exitosa (no gate automático, pero sí flujo UI).

5.7 Logs, almacenamiento y eventos

RF-22 Persistencia de conversación:

- No es obligatorio guardar audio bruto.
- Se guarda la conversación completa en texto:
  - Mensajes del usuario (transcripción o texto)
  - Respuestas del bot
  - Metadatos de ejecución (proveedor, latencia, estados)
- “Mayor información posible del usuario” dentro del alcance v1:
  - Identificador OAuth, proveedor OAuth, user-agent, timestamps, etc.
  - No hay tratamiento especial de PII.

RF-23 Eventos (en BD relacional)
Eventos requeridos:

- conversation_started
- stt_completed
- retrieval_hit
- llm_error
- tts_first_audio

Eventos adicionales recomendados (mínimos operativos):

- user_authenticated
- conversation_ended
- retrieval_miss
- stt_failed, tts_failed
- llm_started, llm_first_token, llm_completed
- tts_started, tts_completed
- barge_in
- network_degraded, fallback_text_only
- kb_index_started, kb_index_completed, kb_index_failed
- bot_published, bot_unpublished
- prompt_updated
- api_key_updated (auditoría)

Campos mínimos por evento:

- event_id, event_name, timestamp
- tenant_id, bot_id, conversation_id
- user_id (OAuth subject), auth_provider
- stage, status, latency_ms (si aplica)
- provider (si aplica)
- error_code, error_message (si aplica)
- payload_json (extensible)

6) Requisitos no funcionales

6.1 Latencia y SLOs (medibles)

Se medirán p50 y p95 para:

- time_to_first_partial_transcript
- time_to_final_transcript
- time_to_first_token
- time_to_first_audio
- end_to_end_turn_time

Los umbrales (ms) se definen como configuración/objetivo operativo del despliegue (SaaS o Standalone).

6.2 Escalabilidad y modos de despliegue

- **SaaS:** multi-tenant con aislamiento lógico por tenant/bot.
- **Standalone:** despliegue en infraestructura de tercero vía containers (ej. docker-compose/helm). Debe soportar configuración por variables de entorno/secret manager.

6.3 Persistencia

- BD relacional para configuración, conversaciones y eventos.
- Vector DB para embeddings de KB (editable desde plataforma).
- Debe existir mecanismo de backup/restore recomendado para ambos componentes (documentado para standalone).

6.4 Seguridad (v1)

- TLS en tránsito.
- RBAC estricto en portal.
- Tokens admin (JWT localStorage) según decisión actual.

- OAuth para público.
- Sin PII programática, sin borrado garantizado (“right to be forgotten” fuera de alcance).

7) Arquitectura y componentes (alto nivel)

7.1 Componentes

1. **Portal Admin (GOB-ID)**: gestión de bots, KB, settings, pruebas, publicación.
2. **Public Web App**: UI voz-first + chat, OAuth, streaming.
3. **Backend API**: orquestación conversación, retrieval, logging, RBAC.
4. **STT Adapter** (pluggable): AssemblyAI u otro.
5. **Embedding + Vector Search**: vector DB + top-k.
6. **LLM Adapter** (pluggable).
7. **TTS Adapter** (pluggable): ElevenLabs u otro.
8. **Relational DB**: config, conversaciones, eventos.
9. **Vector DB**: KB embeddings/chunks.

7.2 App Settings (proveedores y llaves)

- API Keys por proveedor (STT/LLM/TTS/VectorDB si aplica).
- Selector de proveedor activo por etapa.
- Sin retries a proveedores.

8) Modelo de datos (mínimo)

- tenants
- admin_users (GOB-ID subject)
- roles, role_bindings
- bots (nombre, idioma, estado publicado, url_slug)
- bot_prompts (master prompt actual)
- bot_voices (catálogo/IDs por proveedor, voz activa)
- kb_documents (solo .txt, metadata)

- kb_chunks (opcional si se persisten chunks)
- conversations
- messages (user/assistant, texto, timestamps, latencia)
- events (telemetría)
- app_settings (API keys, proveedor activo, defaults globales)
- audit_log (cambios de config sensibles)

9) Extensibilidad futura: trámites e integración con APIs

- Se prevé un módulo de herramientas para conectar APIs:
  - **Function calls** del LLM y/o **MCP Server**.
- En v1 solo se define el “hook” arquitectónico:
  - Registro de herramientas por bot.
  - Permisos para habilitar herramientas.
  - Logging de tool calls como eventos.
- No se implementan trámites reales en esta versión (solo base para integración).

10) Criterios de aceptación (MVP)

- Un admin (GOB-ID) puede crear bot, cargar KB .txt, configurar master prompt, elegir voz/idioma, ejecutar “Probar” y publicar.
- El link público muestra UI estilo call (similar a la imagen) y permite abrir chat.
- Usuario final debe hacer OAuth (Google/Microsoft) para enviar mensajes.
- Conversación por voz:
  - VAD funciona, barge-in corta TTS y toma turno del usuario.
  - Streaming: transcripción parcial, respuesta parcial y TTS streaming.
  - Fallback a texto si falla audio/red.
- RAG grounded-only:
  - top-k retrieval en KB .txt.

    - Si no hay hit → “no sé”.
- Se guarda conversación completa (texto) y eventos mínimos en BD.
- App Settings permite configurar API Keys por etapa y cambiar proveedores sin redeploy del frontend.

11) Parámetros configurables recomendados (iniciales)

- top_k (default sugerido: 5)
- silence_end_ms (p.ej. 1200ms)
- no_speech_timeout_ms (p.ej. 8000ms)
- max_utterance_ms (p.ej. 30000ms)
- toggles: enable_barge_in, enable_tts, enable_typing_sound, enable_text_only_fallback